

UNIVERSIDAD DE COSTA RICA

SEDE GUANACASTE

Multimedios

Proyecto: Aplicación para Guía Turística - Primer entregable

Estudiantes

Juan Carlos Jiménez Castrillo - C33980 - **Líder / Arquitecto de componentes**

Joshua Obando Gonzalez - C35652 - **Desarrollador de componentes de navegación**

Demian Ramírez Sandoval - C36462 - **Desarrollador de componentes de destino**

Maylo Daring Parra Aguirre - C35880 - **Productor multimedia**

Kener Josué Sosa Rodríguez - C37730 - **Diseñador UI/UX**

Profesor

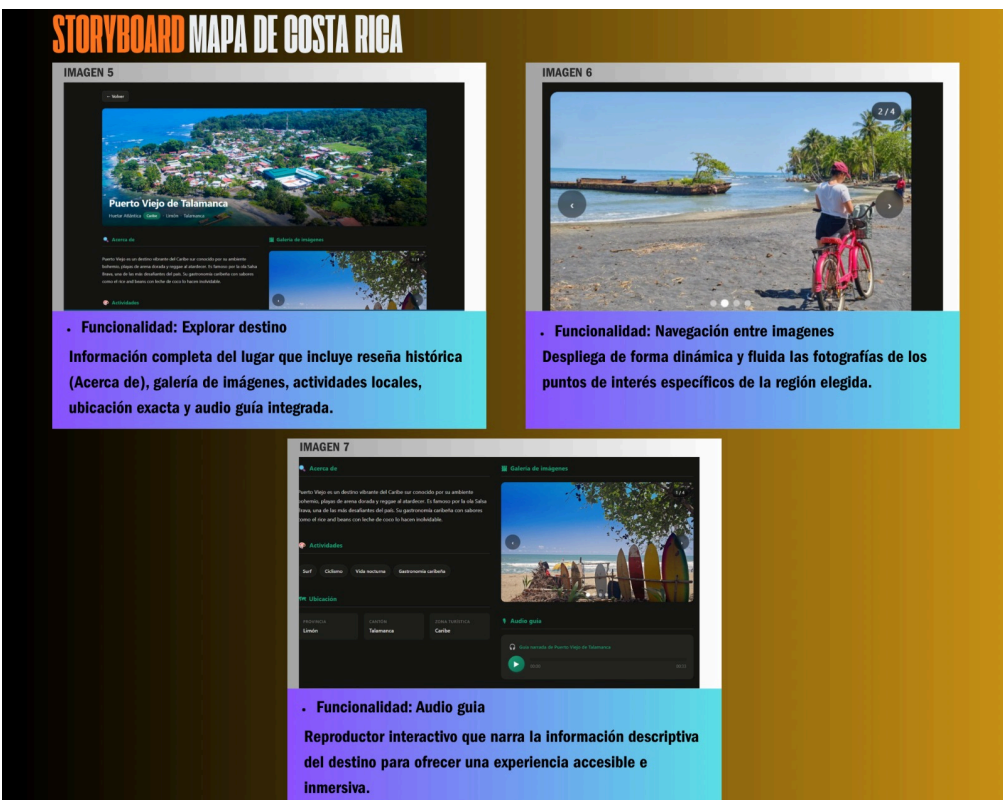
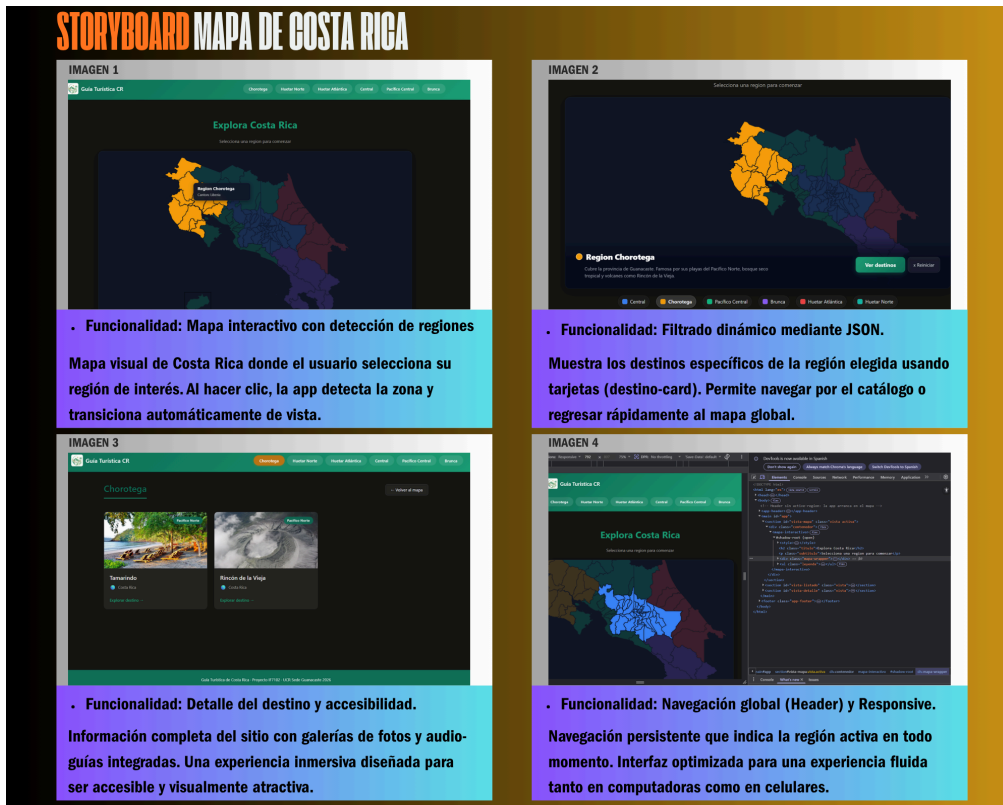
Iván Alonso Chavarria Cubero

Fecha: 04 de junio 2026

ÍNDICE

1. Storyboards.....	3
2. Wireframes.....	4
3. Diagrama de componentes:.....	4
4. Paleta de colores.....	5
5. Tipografía.....	5
6. Decisiones de diseño.....	7

1. Storyboards

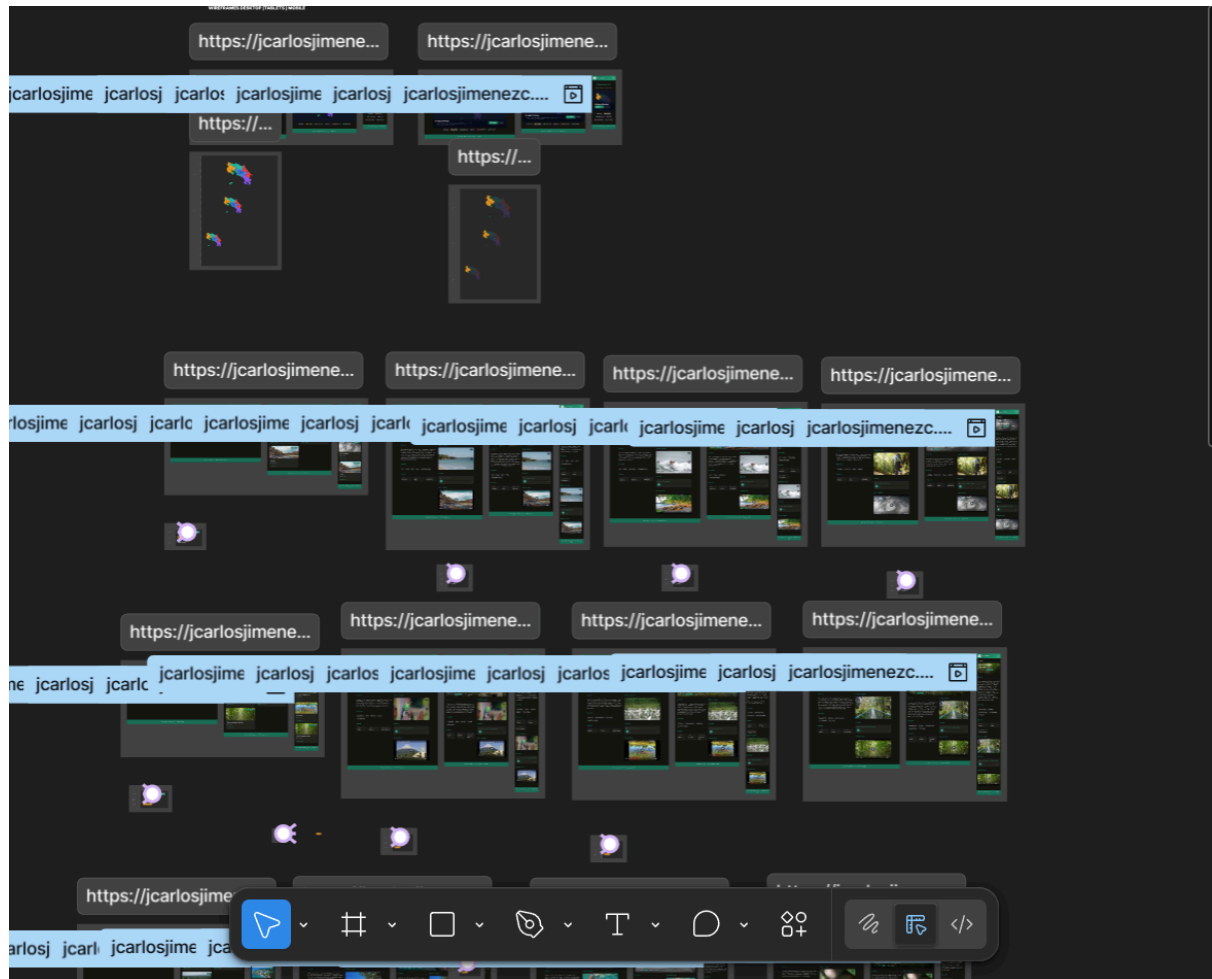


2. Wireframes

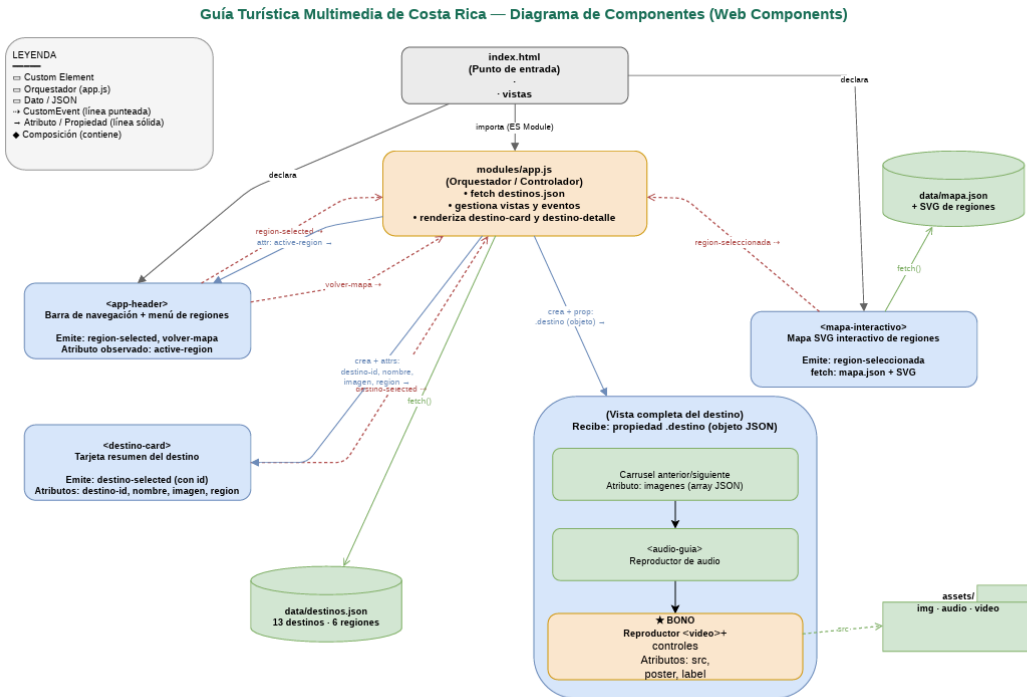
El siguiente link redirige a el proyecto figma donde se realizaron los wireframe

DESKTOP|TABLET|MOBILE: [Figma](#)

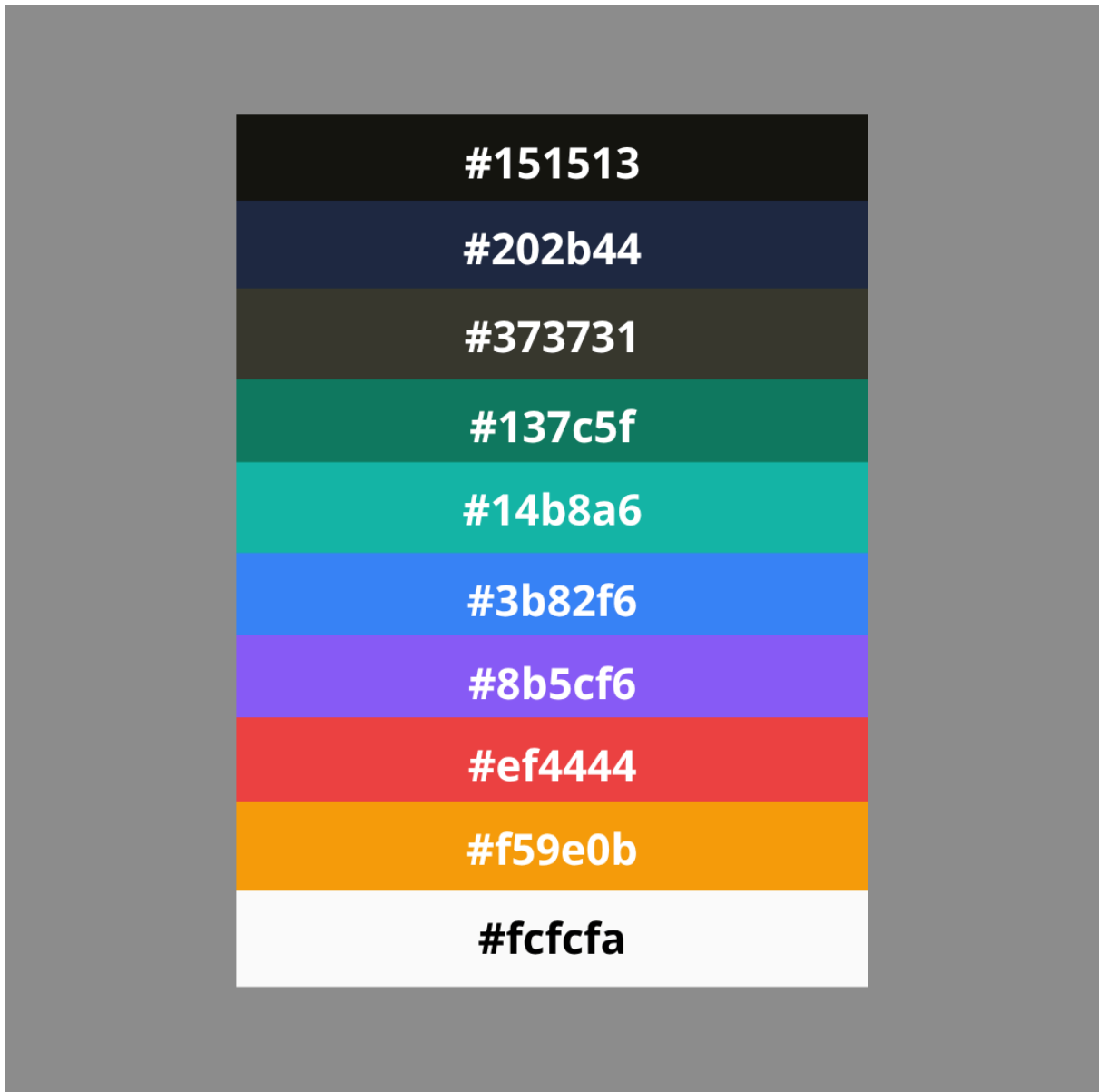
https://www.figma.com/design/Wm2yfyiK6j29E9W71QwF2u/WIRE_FRAMES_GU%C3%8DA_TURISTICA?node-id=0-1&t=W2CSO3JplhEauWyf-1



3. Diagrama de componentes:



4. Paleta de colores



5. Tipografía

La tipografía de la **Guía Turística de Costa Rica** no es solo una elección estética, sino una decisión de ingeniería orientada a la eficiencia y la inclusión.

1. El concepto "System Font Stack" (Pila Nativa)

En lugar de forzar al navegador a descargar archivos de fuentes externos (lo que consumiría datos móviles y tiempo de carga), el código utiliza las fuentes que ya residen en el dispositivo del usuario.

- **Carga Instantánea:** Al eliminar las peticiones HTTP a servidores externos como Google Fonts, el texto es visible desde el milisegundo cero (evitando el efecto de parpadeo de texto invisible).
- **Eficiencia Energética:** Menos descargas significan un menor consumo de batería en dispositivos móviles, lo cual es vital para turistas explorando destinos.

2. Anatomía de la Pila (--font-principal)

La jerarquía de fuentes en el CSS (-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif) garantiza una experiencia optimizada por plataforma:

- **-apple-system / BlinkMacSystemFont:** Invoca la fuente *San Francisco* en dispositivos Apple, diseñada para máxima claridad en pantallas Retina.
- **"Segoe UI":** El estándar de Microsoft para Windows, optimizado para el renderizado de subpíxeles en monitores de escritorio.
- **Roboto:** La fuente nativa de Android, diseñada específicamente para ser legible en pantallas pequeñas y bajo luz solar directa.
- **Helvetica/Arial:** Actúan como redes de seguridad universales para asegurar que el diseño nunca se rompa.

3. Psicología del Estilo Sans-Serif

- **Modernidad y Limpieza:** La ausencia de remates (serifas) proyecta una imagen de vanguardia tecnológica, coherente con una aplicación moderna de Single Page.
- **Reducción de Fatiga Visual:** Las formas geométricas simples de las letras sans-serif facilitan el escaneo visual rápido, permitiendo al usuario encontrar nombres de destinos sin esfuerzo.

4. Arquitectura de Jerarquía y Pesos

El uso de font-weight de **500 a 700** establece una estructura de información clara:

- **Nivel de Autoridad (700 - Bold):** Aplicado a #titulo-region y el logo en el app-header. Crea un anclaje visual que le dice al usuario exactamente dónde se encuentra.
- **Nivel de Interacción (600 - Semi-bold):** Utilizado en las tarjetas (destino-card) y pestañas de navegación. Resalta los elementos que "reaccionan" al tacto o clic.
- **Nivel de Lectura (400 - Normal):** Configurado con un line-height: 1.6 en el cuerpo general. Este espaciado generoso es clave para que las descripciones largas de los destinos no se sientan densas o difíciles de procesar.

5. Accesibilidad y Adaptabilidad

- **Escalado Responsivo:** Gracias al uso de unidades relativas (rem), la tipografía se ajusta automáticamente si el usuario tiene configuraciones de texto grande en su celular por motivos de visión.
- **Contraste:** La tipografía se apoya en la paleta de colores para mantener un ratio de contraste alto, asegurando que el texto sea legible incluso bajo condiciones de mucha luz en exteriores.

6. Decisiones de diseño

6.1. Arquitectura Web Components Nativos

El proyecto adopta exclusivamente la API nativa de Web Components del navegador, alineado con la restricción del curso de no utilizar frameworks externos. Esta decisión implica el uso de Custom Elements v1, Shadow DOM v1, HTML Templates y ES Modules para construir la interfaz. Se definieron seis componentes reutilizables, cada uno encapsulado en su propio archivo .js: <app-header>, <mapa-interactivo>, <destino-card>, <destino-detalle>, <galeria-imagenes> y <audio-guia>. El Shadow DOM en modo open garantiza el aislamiento completo de estilos y estructura interna, evitando colisiones CSS globales y permitiendo que cada componente gestione su propio renderizado y lógica de interacción sin dependencias externas. Esta arquitectura promueve la reutilización y la mantenibilidad del código a largo plazo.

6.2. Comunicación Entre Componentes y Arquitectura SPA

La navegación se implementa como una Single Page Application sin recarga de página, gestionando tres vistas principales (mapa, listado y detalle) mediante clases CSS que controlan la visibilidad. La comunicación entre componentes se resuelve mediante CustomEvents con las opciones bubbles: true y composed: true, permitiendo que los eventos atraviesen los límites del Shadow DOM. Los eventos documentados incluyen region-seleccionada (emitido desde el mapa), region-selected (emitido desde el header), destino-selected (emitido desde las tarjetas) y volver-mapa (emitido desde el logo). El archivo app.js actúa como orquestador central: recibe los eventos, filtra los datos del JSON, instancia los componentes necesarios y alterna entre vistas usando window.scrollTo para mantener la experiencia fluida.

6.3. Carga Dinámica de Datos y Separación de Responsabilidades

Los datos de los destinos se mantienen en un archivo `destinos.json` independiente, siguiendo una estructura bien definida con campos como `id`, `nombre`, `region`, `descripcion`, `imagen_portada`, `galeria`, `audio`, `video`, `actividades`, `lat` y `lng`. El módulo `app.js` carga este JSON mediante `fetch()` al iniciar la aplicación y almacena los datos en una variable global. Al seleccionar una región, se filtran dinámicamente los destinos correspondientes y se renderizan inyectando instancias de `<destino-card>` en el DOM con atributos serializados. Esta separación entre datos y presentación permite escalar el contenido sin modificar el código fuente de los componentes, cumpliendo con el principio de responsabilidad única y facilitando la gestión del contenido multimedia.

6.4. Integración Multimedia

Audio: El componente `<audio-guia>` implementa un reproductor personalizado que no depende de controles nativos visuales. Utiliza una instancia de `Audio()` en JavaScript y resuelve un problema técnico específico: los archivos MP3 generados con TTS no incluyen headers Xing/ID3, lo que impide la detección de duración mediante el evento `loadedmetadata`. La solución consiste en cargar el audio mediante `fetch()`, convertirlo a Blob y generar una URL de objeto (`URL.createObjectURL()`), lo que permite al navegador calcular correctamente la duración. La barra de progreso se actualiza con `requestAnimationFrame` durante la reproducción y soporta seeking mediante clic. Se incluyen atajos de teclado (flechas para retroceder/avanzar 5 segundos) y se limpian las URLs de objeto en `disconnectCallback` para evitar fugas de memoria.

Imágenes: Todas las imágenes del proyecto utilizan el formato WebP para optimizar el peso sin sacrificar calidad. Se aplica el atributo `loading="lazy"` en las imágenes de portada y galería para diferir la carga hasta que el elemento sea visible en el viewport. El componente `<galeria-imagenes>` implementa un carrusel con navegación anterior/siguiente, indicadores visuales (dots), contador de posición y soporte para navegación por teclado (flechas izquierda/derecha). Se incluye manejo de errores mediante el evento `onerror` que reemplaza imágenes rotas por un placeholder visual, asegurando que la interfaz no se rompa ante contenido faltante.

Video: La estructura del JSON ya incluye la propiedad `video` para cada destino (referenciando archivos en `assets/video/`), preparando la arquitectura para la integración de un componente `<video-destino>`. Siguiendo el mismo patrón de diseño que `<audio-guia>`, este componente futuro utilizará el elemento `<video>` nativo dentro del Shadow DOM, con

atributos observados para src y poster, controles personalizados encapsulados y gestión de eventos propios. La decisión de diseño fue estructurar los datos desde la fase inicial para que la incorporación del reproductor de video no requiera modificaciones en el modelo de datos ni en el orquestador principal.

6.5. Mapa Interactivo SVG

El componente <mapa-interactivo> renderiza un SVG vectorial de los cantones de Costa Rica. Cada path del SVG se asocia a una región turística mediante un mapeo definido en mapa.json. La interacción se resuelve a nivel de cantones: al hacer hover, el componente identifica la región correspondiente, aplica colores diferenciados por zona y muestra tooltips con position: fixed para evitar problemas de overflow. Al seleccionar una región, se calcula automáticamente el viewBox óptimo usando getBBox() de los cantones pertenecientes a esa región, generando un zoom dinámico sin librerías externas. Se añade un margen de seguridad de 40px en el viewBox para garantizar que todos los cantones sean visibles. Un panel overlay inferior aparece con animación slideUp al seleccionar una región, mostrando el nombre y una descripción breve. En dispositivos móviles, la descripción del panel se oculta para optimizar el espacio y los botones se apilan verticalmente.

6.6. Diseño Responsive y Mobile-First

El diseño parte desde la menor resolución hacia la mayor, utilizando un sistema de variables CSS globales para colores, espaciado, sombras y transiciones. Estas variables atraviesan el Shadow DOM, permitiendo coherencia visual entre componentes encapsulados. El grid de destinos utiliza repeat(auto-fill, minmax(280px, 1fr)) para reorganizarse automáticamente según el ancho disponible. Los breakpoints definidos son: escritorio (>1024px), tablet (769px–1024px) y móvil (<768px). El header es position: sticky y en móvil muta a un menú hamburguesa que alterna la clase abierta para desplegar la navegación en columna. En el detalle de destino, el layout pasa de dos columnas en escritorio a una sola columna en móvil. Se incluye soporte nativo para modo oscuro mediante @media (prefers-color-scheme: dark), redefiniendo todas las variables CSS sin tocar código JavaScript.

6.7. Accesibilidad

La aplicación prioriza la inclusión mediante un conjunto de decisiones técnicas. Se utiliza la clase `.sr-only` en `global.css` para ocultar elementos visualmente manteniéndolos disponibles para lectores de pantalla. Se aplican atributos ARIA extensivos: `aria-label` en botones sin texto visible, `aria-current="page"` en la navegación activa, `aria-expanded` en el menú hamburguesa, `role="button"` y `role="slider"` en controles interactivos, y `aria-live="polite"` en el contador de la galería. La navegación por teclado es completa: las tarjetas responden a Enter/Espacio, la galería a flechas izquierda/derecha, y el audio a flechas para seeking. Se implementa `prefers-reduced-motion` para respetar la preferencia de usuario de reducir animaciones. Los estilos `:focus-visible` están definidos en todos los componentes para garantizar que el foco del teclado sea visible sin interferir con el uso del mouse. Finalmente, el System Font Stack (ya detallado en la sección de Tipografía) elimina peticiones HTTP a servidores de fuentes, mejorando la velocidad de carga y reduciendo el consumo de datos móviles, lo cual es crítico para turistas en campo.